

Quiz #1

April 2, 2025

Student ID:

Name:

Q1. What is the main purpose of operating system? (1point)

OS provides environment and OS manages system resources.
or OS is in charge of making sure the system operates correctly and efficiently.

Q2. What is the relationship between system calls and dual mode? (1point)

A system call is the mechanism that allows a user-mode program to request a service from the OS in kernel mode, such as file access or memory allocation. System calls rely on dual mode to safely switch from user mode to kernel mode, allowing user programs to access operating system services without compromising system security.

Q3. Process provides two key abstractions, logical control flow and private address space. What does two key abstraction mean? Explain it in terms of functionality of OS. (1point)

- Logical Control Flow: Each program seems to have exclusive use of the CPU. This means each process runs independently and continuously, even though the CPU may be shared among multiple processes. This abstraction is enabled by context switching and scheduling services (or multitasking).

- Private Address Space: Each program seems to have exclusive use of main memory. This means each process has its own memory space that other processes cannot access. The OS ensures memory protection and isolation, allowing processes to run without interfering with each other's data. This abstraction is implemented using virtual memory

Q4. Below is the output of a simple shell program. Complete the source code to produce the output shown below. (1point)

* Output of a simple shell program

```
yunmin@mcnl:~/quiz01$ ./myshell
mysh> ls
hello.c myshell myshell.c
mysh> ls -l
total 28
-rw-rw-r-- 1 yunmin yunmin  83  4월  1 14:22 hello.c
-rwxrwxr-x 1 yunmin yunmin 16568 4월  1 13:49 myshell
-rw-rw-r-- 1 yunmin yunmin 1439 4월  1 13:48 myshell.c
mysh> cat hello.c
#include <stdio.h>
int main() {
    printf("Hello Handong!\n");
    return 0;
}
```

```

mysh> gcc hello.c -o hello
mysh> ls
hello hello.c myshell myshell.c
mysh> ./hello
Hello Handong!
mysh> quit
Shell exited
yunmin@mcnl:~/quiz01$

```

```

#define MAX_LINE 1024
#define MAX_ARGS 64

int main() {
    char line[MAX_LINE];
    char *args[MAX_ARGS];

    while (1) {
        printf("mysh> ");
        fflush(stdout);

        if (!fgets(line, MAX_LINE, stdin)) {
            perror("fgets failed");
            break;
        }

        line[strcspn(line, "\n")] = '\0';

        if (strcmp(line, "quit") == 0)
            break;

        int argc = 0;
        char *token = strtok(line, " ");
        while (token != NULL && argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }
        args[argc] = NULL;

        // TODO: Implement from here!
        // Hint: Use execvp() to run a new program and refer to Reference!
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork failed");
        } else if (pid == 0) {
            if (execvp(args[0], args) == -1) {
                perror("execvp failed");
            }
            exit(1);
        } else {
            wait(NULL);
        }

    }
    printf("Shell exited.\n");
    return 0;
}

```

Q5. Explain the difference in the output between the two codes below. Provide a reason for your answer. (1point)

* Code #1

```

void main() {
    pid_t pid[N];
    int i, child_status;

```

```

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0)
            while(1);

    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}

```

* Code #2

```

void main() {
    pid_t pid[N];
    int i;
    int child_status;

    signal(SIGINT, int_handler);
    for (i = 0; i < N; i++) {
        if ((pid[i] = fork()) == 0)
            while(1);
    }

    for (i = 0; i < N; i++) {
        printf("Killing process %d\n", pid[i]);
        kill(pid[i], SIGINT);
    }

    for (i = 0; i < N; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminated abnormally\n", wpid);
    }
}

void int_handler(int sig){
    printf("Process %d received signal %d\n", getpid(), sig);
    exit(0);
}

```

Code#1 prints out “Child <pid> terminated abnormally” since child process terminates abnormally by SIGINT.
 Code#2 prints out “Child <pid> terminated with exit status 0” since child process terminated normally by exit().